

Module 6 — Graphs & Structured Facts

From raw text to verifiable knowledge

🎯 Learning goals

- Use **structured outputs** (Pydantic + JSON Schema) for reliable typed data.
- Build a **fact extraction pipeline** with provenance and confidence.
- Construct a **knowledge graph** from entities and relationships.
- Implement **grounded QA** that cites source documents.
- Expose the graph as **MCP tools** an agent loop can call.



Structured Outputs

Typed fields the model must follow

Pydantic model as output schema

```
from pydantic import BaseModel, Field

class Fact(BaseModel):
    subject: str = Field(description="The entity the fact is about")
    predicate: str = Field(description="The relationship or action")
    object: str = Field(description="The target entity or value")
    source_text: str = Field(description="Original sentence")
    confidence: float = Field(ge=0.0, le=1.0)

class FactList(BaseModel):   # top level must be an object
    facts: list[Fact]
```

- → Field descriptions go into the JSON Schema the model sees
- → Field constraints (ge, le) are part of the schema and re-checked by Pydantic

Structured outputs — the schema is the format

```
response = client.responses.parse(
    model=MODEL,
    reasoning={"effort": "none"},
    text_format=FactList,
    input=[
        {"role": "system", "content": "Extract every factual claim. Rate confidence 0-1."}
        {"role": "user", "content": text},
    ],
)
facts = response.output_parsed.facts if response.output_parsed else []   # None = refusal

# Input:  "Alice manages the infrastructure team."
# facts:  [Fact(subject='Alice', predicate='manages',
#               object='infrastructure team', ...)]
```

- → The API constrains generation to the schema — no missing keys, no wrong types
- → Older code used JSON mode (json_object) + schema in the prompt: valid JSON, but not necessarily yours



Demo — Structured extraction

Switch to terminal: python demo/01_extraction.py



Knowledge graphs

Entities, relationships, and multi-hop traversal

Knowledge graphs

- **Nodes** = entities (people, organisations, products).
- **Edges** = relationships (manages, works_at, supplies).
- Build from extracted facts: subject → edge → object.
- Traversal: "Who is connected to Acme Corp within 2 hops?"



Using an LLM to build the Graph

The Language model is the graph builder

Using an LLM to build the Graph

- Have the language model read your document and convert it to facts
- Find Subjects, Objects, and Predicates
- Then use the facts to build the graph

From facts to nodes and edges

- Each validated fact is a triple: **subject → predicate → object**.
- **Subject** and **object** become nodes; **predicate** becomes the directed edge.
- *Marquez assessed the Meridian reactor* → `Juno Marquez --[assessed]--> Meridian reactor`.
- Same name = same node, so facts from different logs **join into one graph**.
- Confidence and source text live **on the edge** — every hop stays citable.

⚠ Constraining Vocabulary

- A **controlled vocabulary** for predicates — `predicate: Literal["assessed", "repaired", ...]`.
- **Canonicalise entity names** before adding them, so aliases collapse to one node.

Building the graph

```
@dataclass
class Entity:
    name: str
    entity_type: str
    attributes: dict[str, str] = field(default_factory=dict)

@dataclass
class Relationship:
    source: str
    target: str
    relation: str

class KnowledgeGraph:
    def __init__(self):
        self.entities: dict[str, Entity] = {}
        self.edges: list[Relationship] = []

    def neighbours(self, name: str) -> list[Relationship]:
        return [e for e in self.edges
                if e.source == name or e.target == name]
```

- → Simple adjacency list — no external graph database needed
- → neighbours() enables BFS/DFS traversal for multi-hop queries

Multi-hop traversal

```
def find_connections(graph, start, max_depth=2):
    visited = set()
    queue = [(start, 0)]
    results = []

    while queue:
        node, depth = queue.pop(0)
        if node in visited or depth > max_depth:
            continue
        visited.add(node)

        for rel in graph.neighbours(node):
            results.append(rel)
            next_node = (rel.target if rel.source == node
                         else rel.source)
            if next_node not in visited:
                queue.append((next_node, depth + 1))

    return results
```

- BFS from the starting entity up to max_depth hops
- Returns all relationships discovered along the way



Demo — Knowledge graph

Switch to terminal: python demo/02_graph.py



The graph as an MCP tool

An MCP server the agent chooses to call

Let the agent walk the graph

- An MCP server turns queries into **tools**: list, neighbours, path, facts within N hops.
- The agent loop is the one from Module 3. The model picks the tool. Your code runs it.
- The server is deterministic. It reads a saved graph — no model call inside the tool.

The MCP server

- Neighbours: list all entities connected to a given entity
- Path: find the shortest path between two entities
- Facts: list all facts about a given entity
- Facts within N hops: list all facts about a given entity

One tool, one graph question

```
@server.tool()
def shortest_path(start: str, end: str) -> str:
    """Shortest chain of facts connecting two entities."""
    ...

# The agent sees the tool list, then:
#   list_entities(query="chief engineer")
#   shortest_path(start="Mira Chen", end="osmium")
```

- → The docstring and the type hints become the schema the model reads
- → stdio: demo 04 spawns graph_server.py, no second terminal



Demo — Graph agent

Switch to terminal: python demo/04_graph_agent.py



Putting it all together

Summary and exercises

Summary

Schema is the contract

Pydantic structured outputs are what you can validate, store, and query.

Every fact needs provenance and a score

A fact without a source is an unsupported claim. Confidence decides what you keep.

Entities and edges make a graph

Multi-hop questions walk relationships, not just similar text.

Grounded QA cites the source

Answer from stored facts, and show where each one came from.

The graph can be a tool

An MCP server answers list, neighbours, and path. The agent decides which to call.

Exercises

01 — Fact extractor: Pydantic schemas and LLM extraction with validation

02 — Knowledge graph: entities, relationships, and multi-hop traversal

03 — Grounded QA: answer questions with citations and confidence scores



Module 6 — Complete

Next: agent memory